

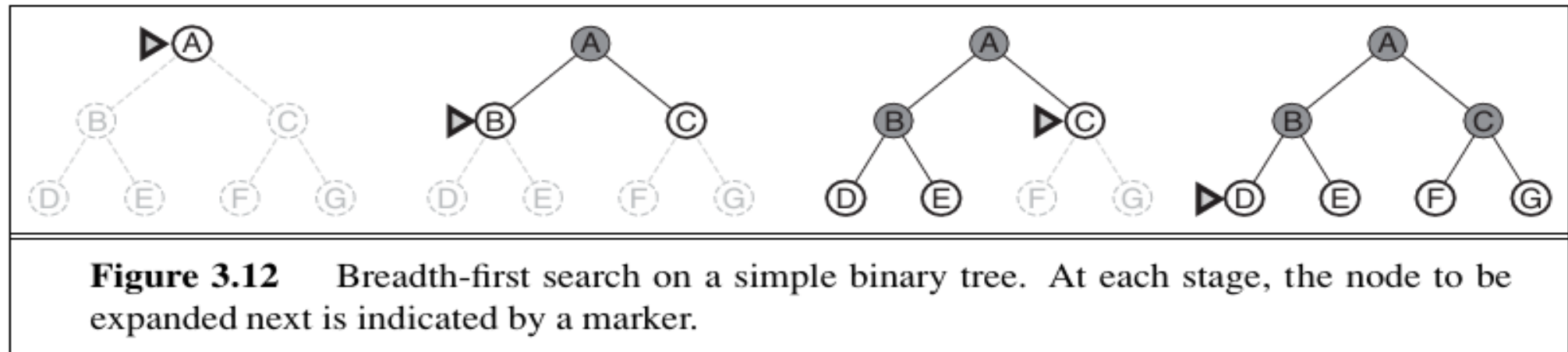
Chapter 4: Searching Techniques

Search

- The process of finding a path to desired goal state from all possible future states.
- There are 2 kinds of search based on whether they use information about the goal.
- Uninformed Search Algorithms : - Algorithms that are given no information about the problem other than its definition.
- Informed Search Algorithms: The algorithms of informed search contain information regarding the goal state. It helps an AI make more efficient and accurate searches.

Breadth First Search

- Breadth-first search is a simple strategy in which the root node is expanded first, then all the successors of the root node are expanded next, then their successors, and so on.
- This is achieved very simply by using a FIFO queue.



```
marked = [False] * G.size()
```

```
def bfs( $G$ ,  $v$ ) :
```

```
    queue = [ $v$ ]
```

```
→ while len(queue) > 0:
```

```
     $v$  = queue.pop(0)
```

```
    if not marked[ $v$ ]:
```

```
        visit( $v$ )
```

```
        marked[ $v$ ] = True
```

```
        for  $w$  in G.neighbors( $v$ ):
```

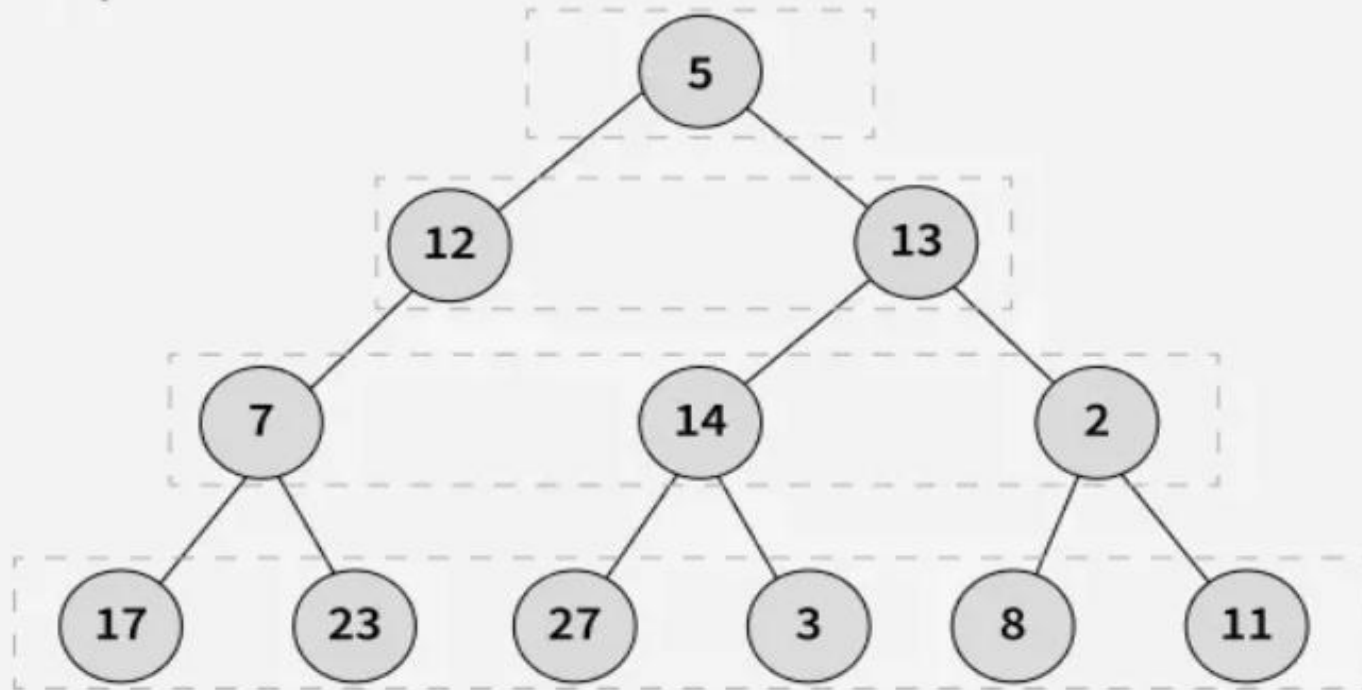
```
            if not marked[ $w$ ]:
```

```
                queue.append( $w$ )
```

Breadth First Search

01
Step

Add the root node to the Queue.



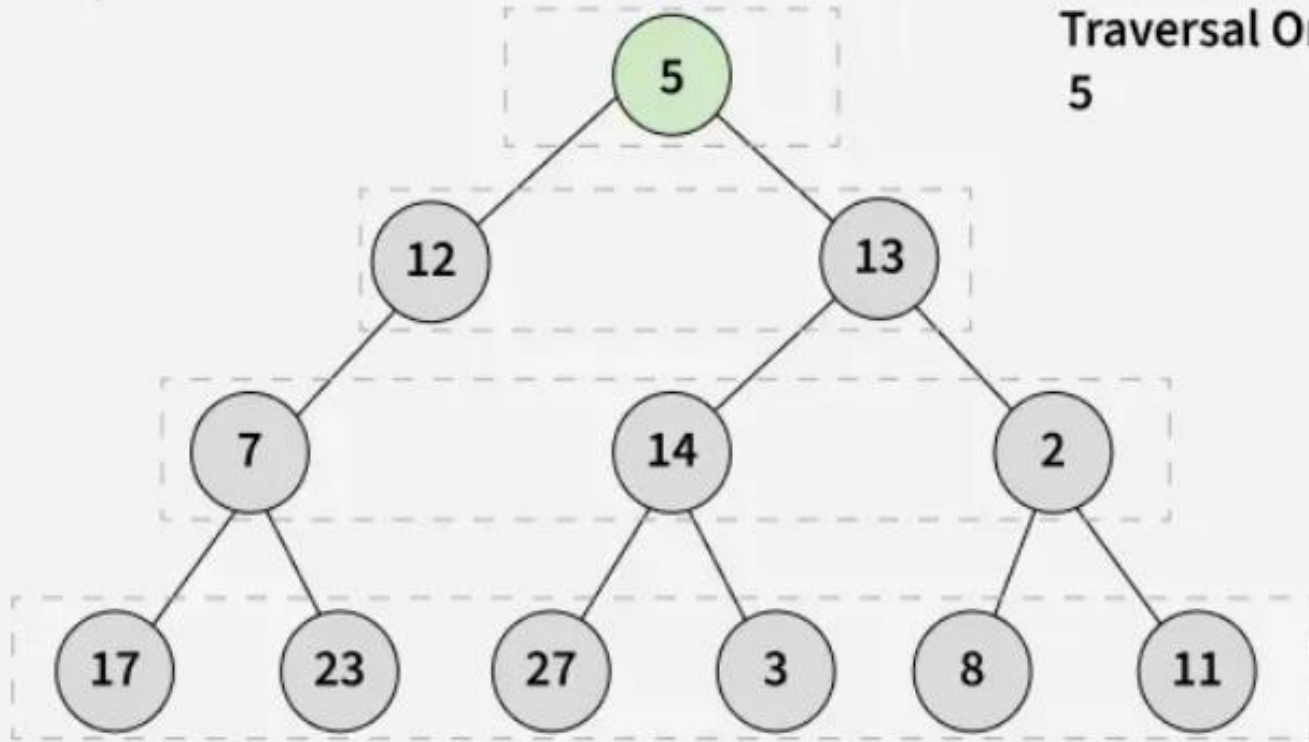
Queue

5		
---	--	--

Level Order Traversal

02
Step

Pop the front element (**5**) from the Queue and add its left child (**12**) and right child (**13**) to the queue respectively.



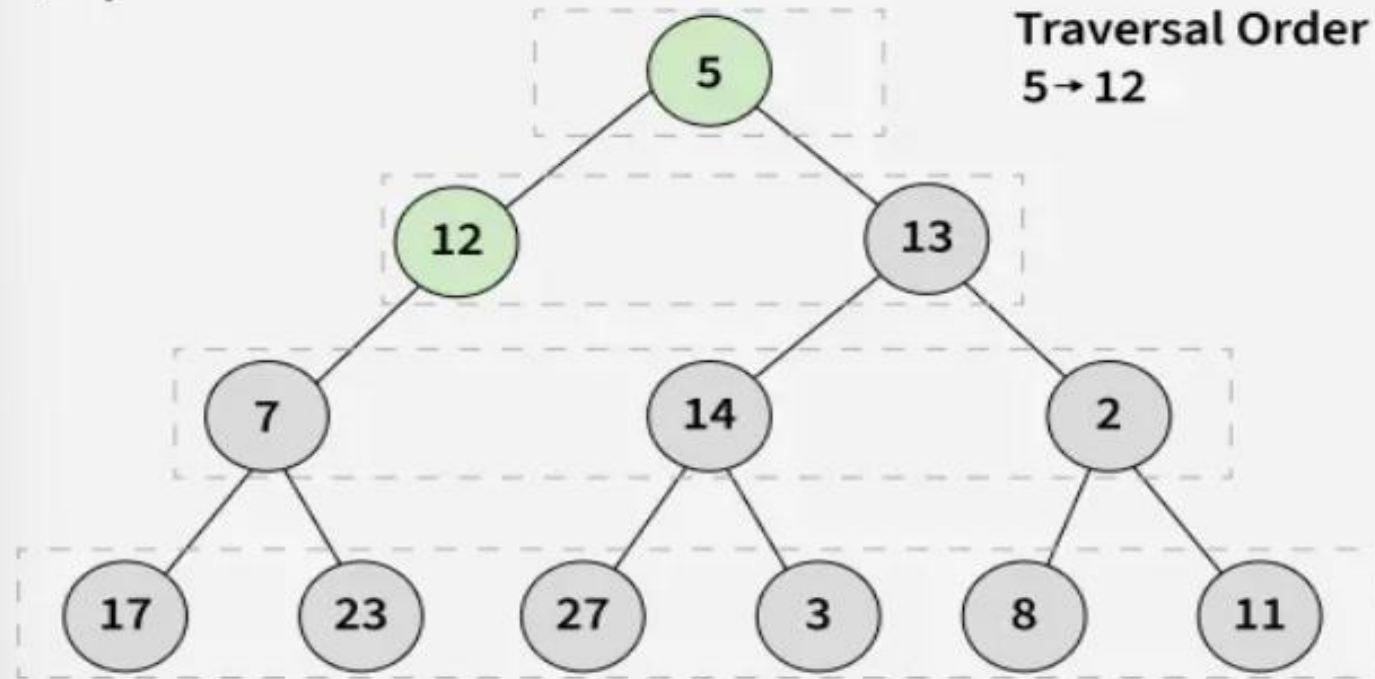
Traversal Order
5

Queue
5 12 13

Level Order Traversal

03
Step

Pop the front element (**12**) from the Queue and add its left child (**7**) to the queue.



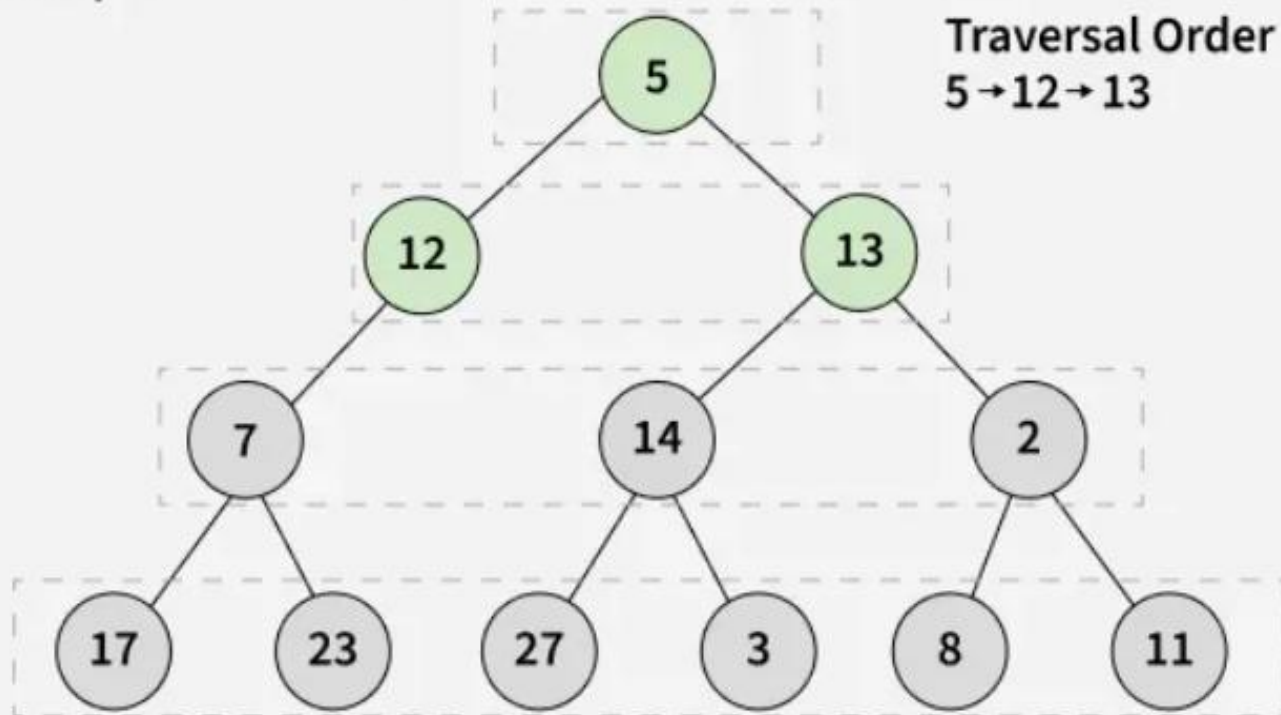
Traversal Order
5 → 12



Level Order Traversal

04
Step

Pop the front element (**13**) from the Queue and add its left child (**14**) and right child (**2**) to the queue respectively.



Traversal Order
5 → 12 → 13

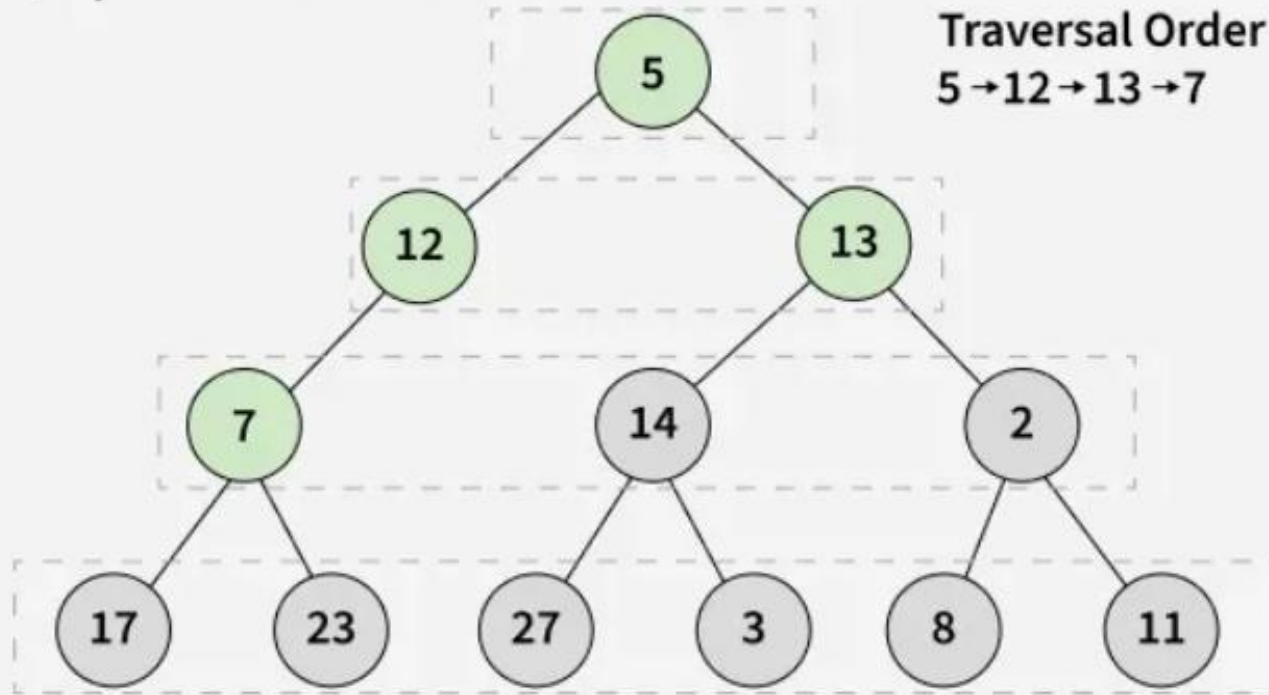
Queue

13	7	14	2	
----	---	----	---	--

Level Order Traversal

05
Step

Pop the front element (**7**) from the Queue and add its left child (**17**) and right child (**23**) to the queue respectively.



Traversal Order
5 → 12 → 13 → 7

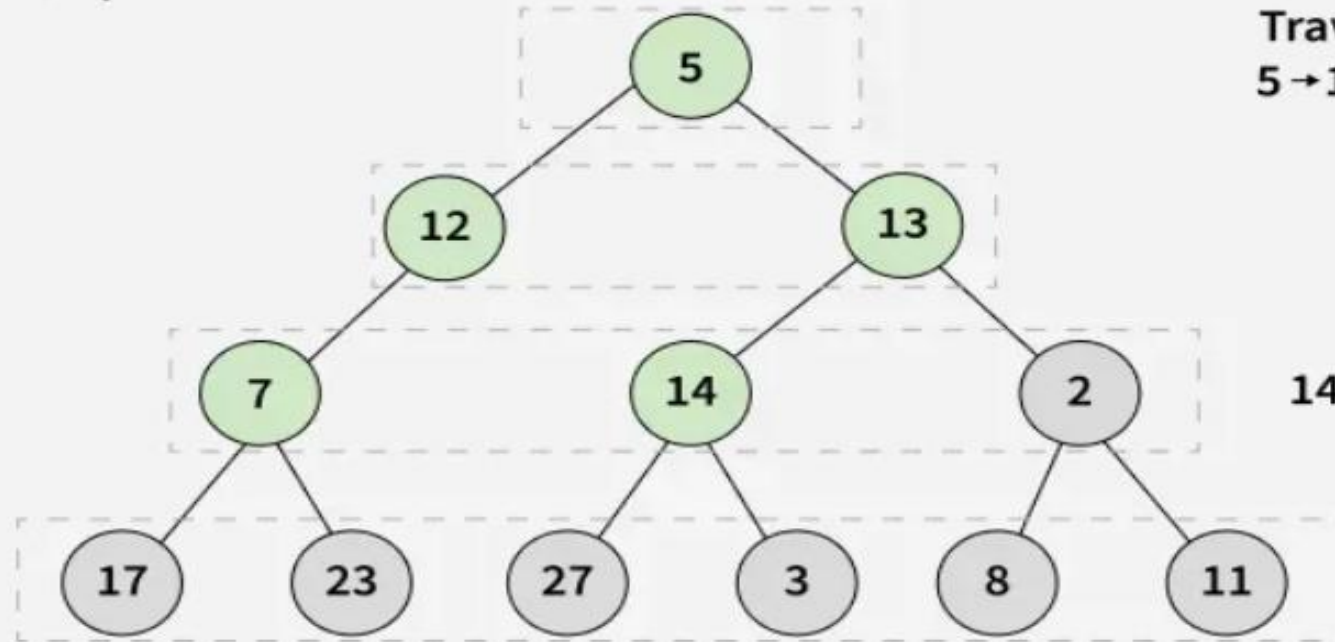
Queue

7 14 2 17 23

Level Order Traversal

06
Step

Pop the front element (**14**) from the Queue and add its left child (**27**) and right child (**3**) to the queue respectively.



Traversal Order
 $5 \rightarrow 12 \rightarrow 13 \rightarrow 7 \rightarrow 14$

Queue

14

2	17	23	27	3	
---	----	----	----	---	--

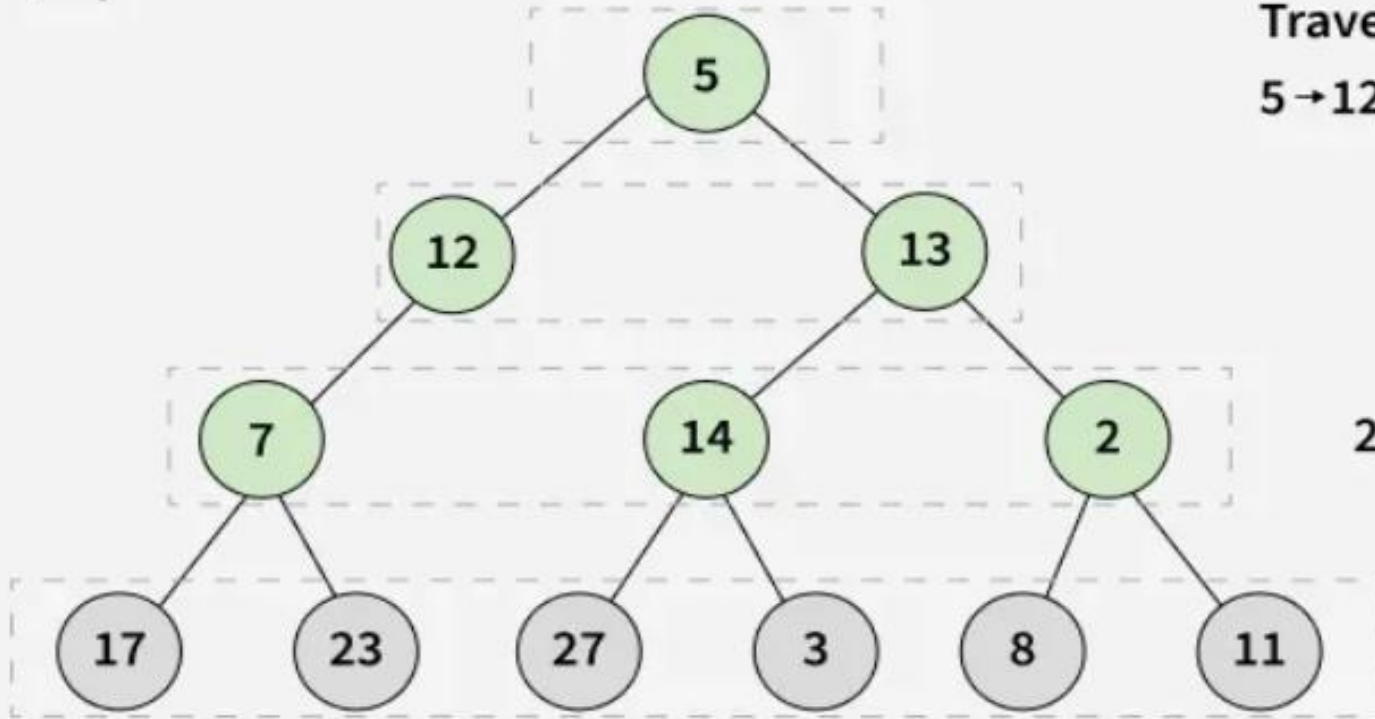
Level Order Traversal

07
Step

Pop the front element (**2**) from the Queue and add its left child (**8**) and right child (**11**) to the queue respectively.

Traversal Order

5 → 12 → 13 → 7 → 14 → 2



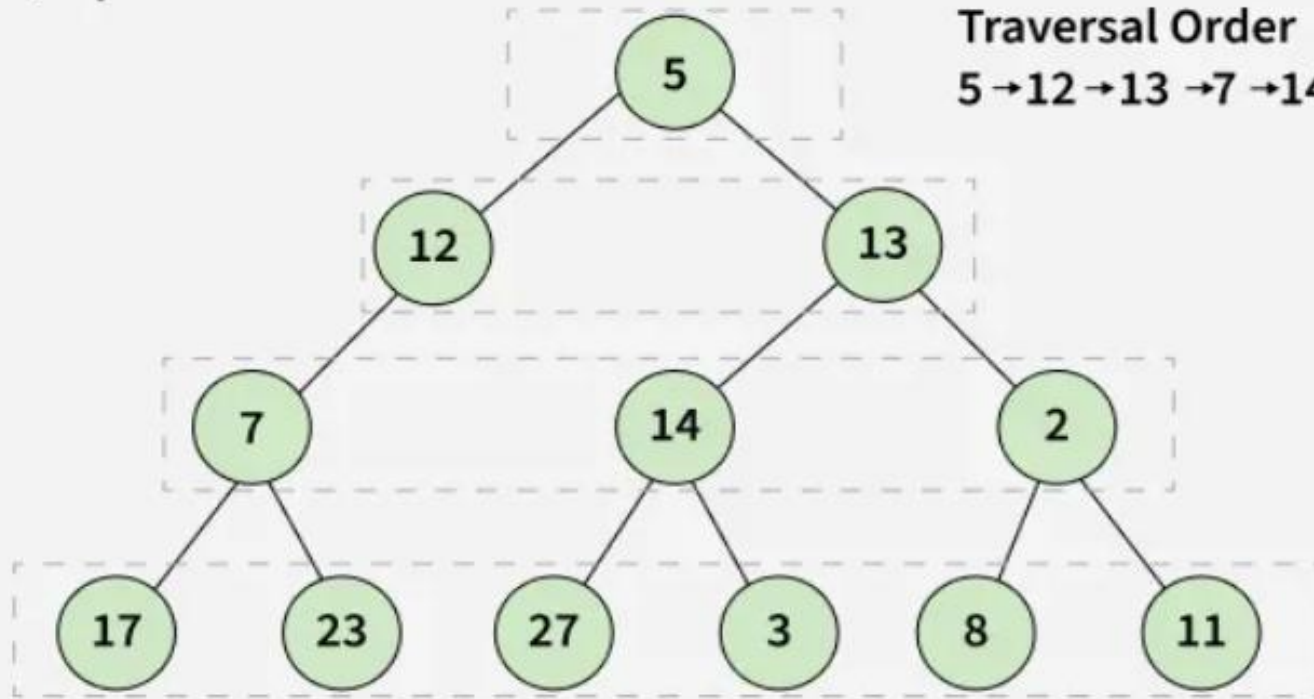
Queue

2	17	23	27	3	8	11	
---	----	----	----	---	---	----	--

Level Order Traversal

08
Step

Similarly pop the rest of the nodes from the queue until the queue is empty.



Traversal Order

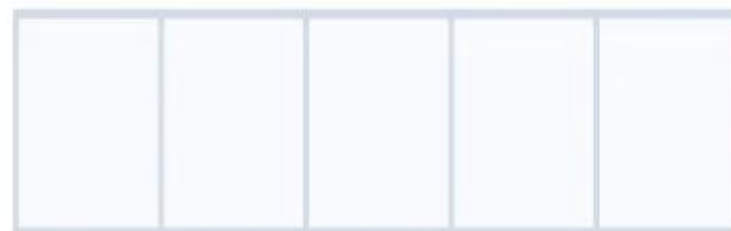
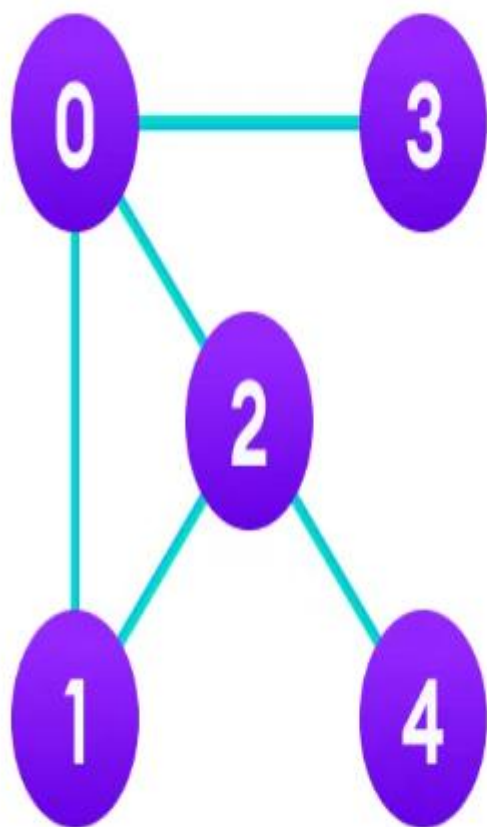
5 → 12 → 13 → 7 → 14 → 2 → 17 → 23 → 27 → 3 → 8 → 11

Level Order Traversal

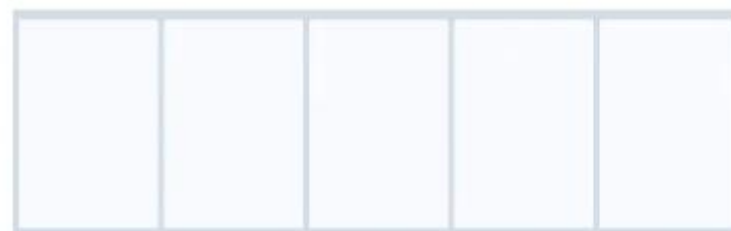
Depth First Search

- Depth First Search (DFS) is a graph traversal method that starts from a source vertex and explores each path completely before backtracking and exploring other paths.
- To avoid revisiting nodes in graphs with cycles, a visited array is used to track visited vertices.
- There can be multiple DFS traversals of a graph according to the order in which we pick adjacent vertices.

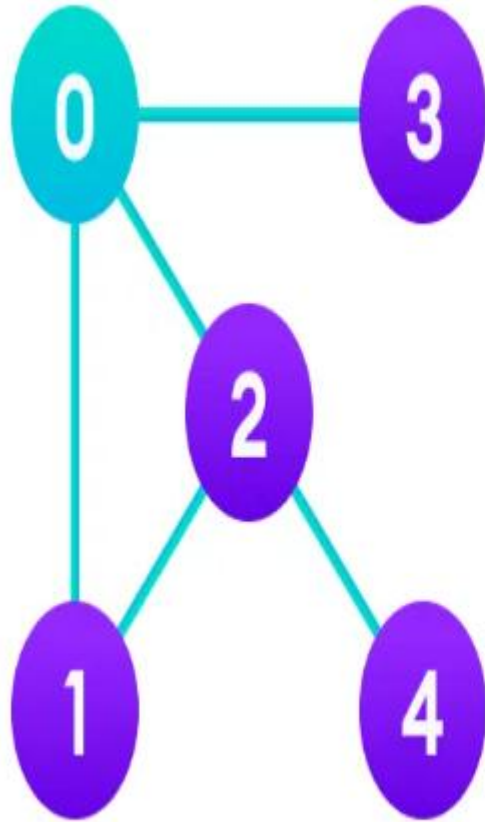
```
marked = [False] * G.size()
def dfs_iter( $G$ ,  $v$ ) :
    stack = [ $v$ ]
    while len(stack) > 0:
         $v$  = stack.pop()
        if not marked[ $v$ ]:
            visit( $v$ )
            marked[ $v$ ] = True
            for  $w$  in  $G$ .neighbors( $v$ ):
                if not marked[ $w$ ]:
                    stack.append( $w$ )
```



Visited



Stack

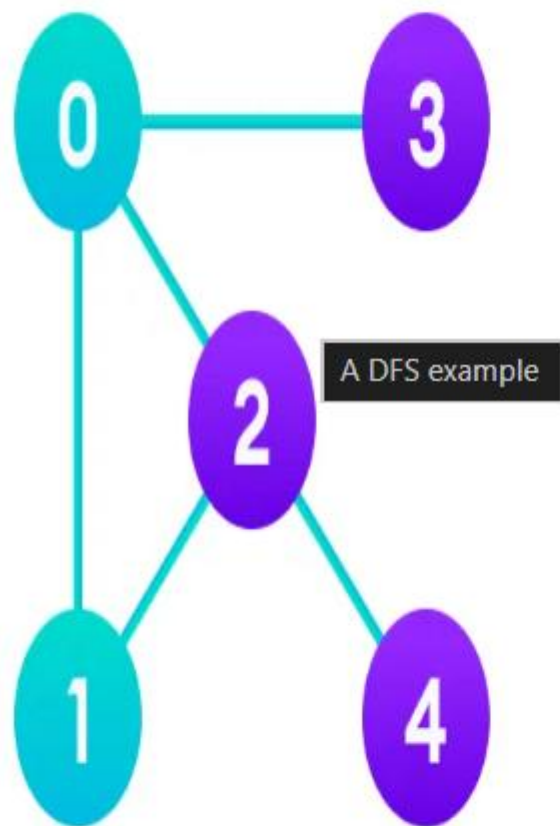


0				
---	--	--	--	--

Visited

3	2	1		
---	---	---	--	--

Stack



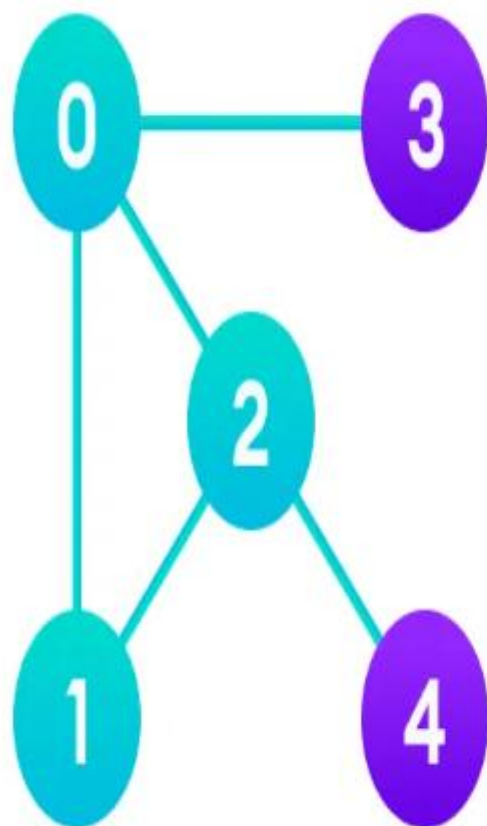
A DFS example

0	1			
---	---	--	--	--

Visited

3	2			
---	---	--	--	--

Stack

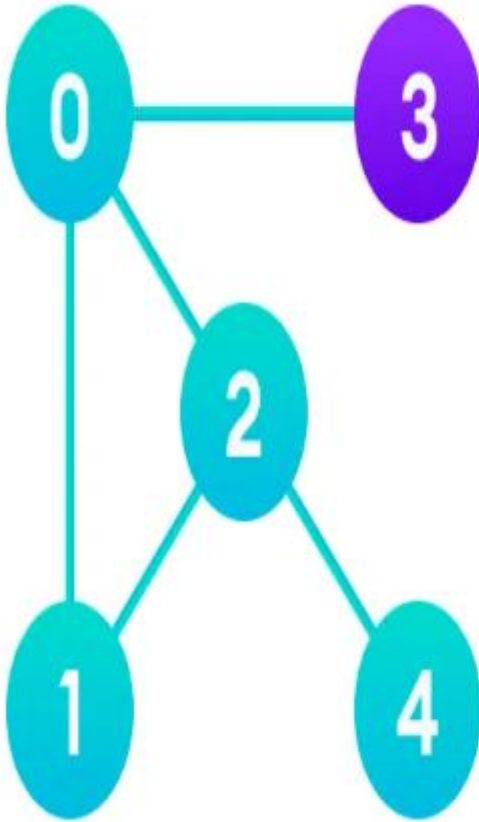


0	1	2		
---	---	---	--	--

Visited

3	4			
---	---	--	--	--

Stack

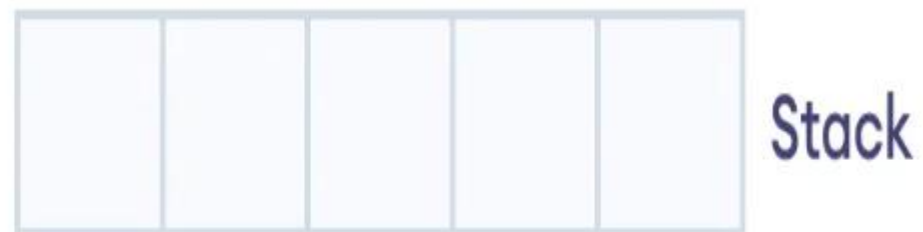
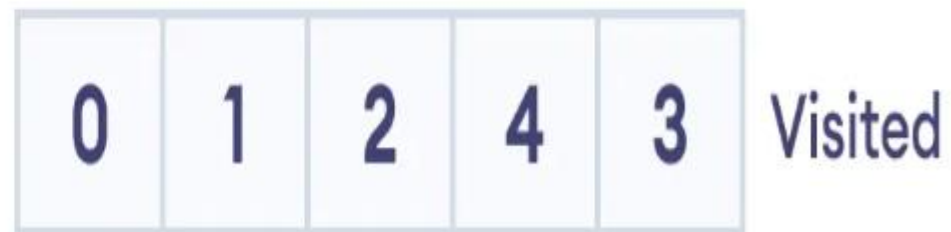
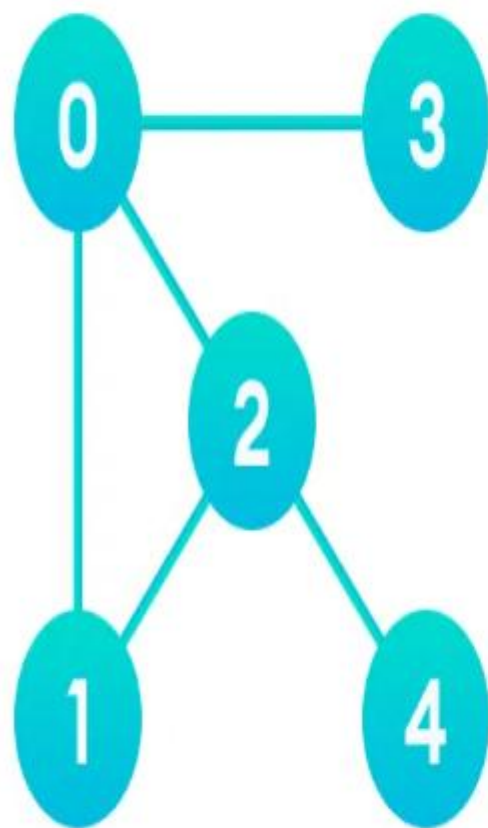


0	1	2	4	
---	---	---	---	--

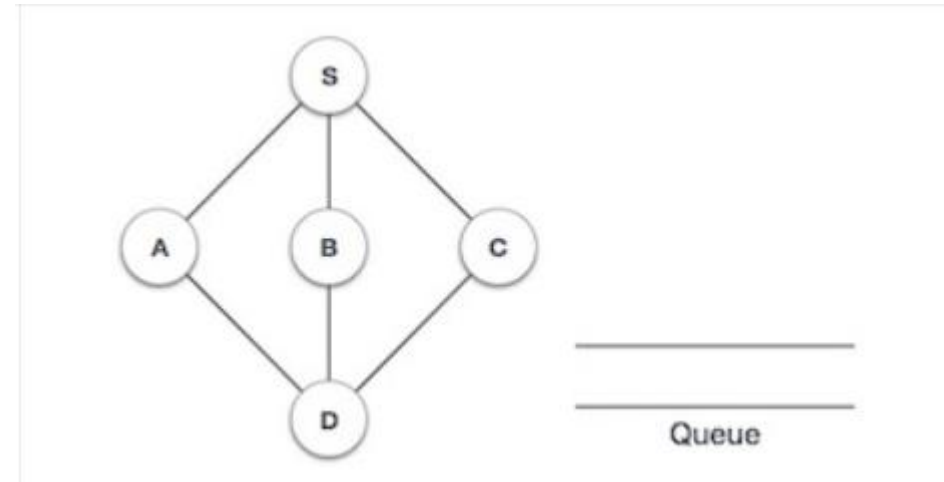
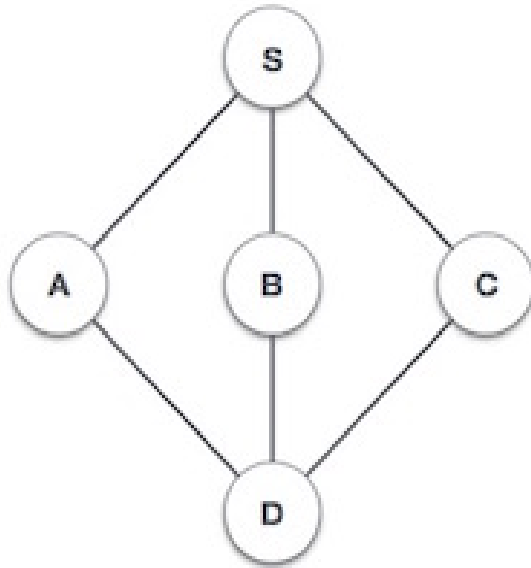
Visited

3				
---	--	--	--	--

Stack



Classwork for BFS and DFS



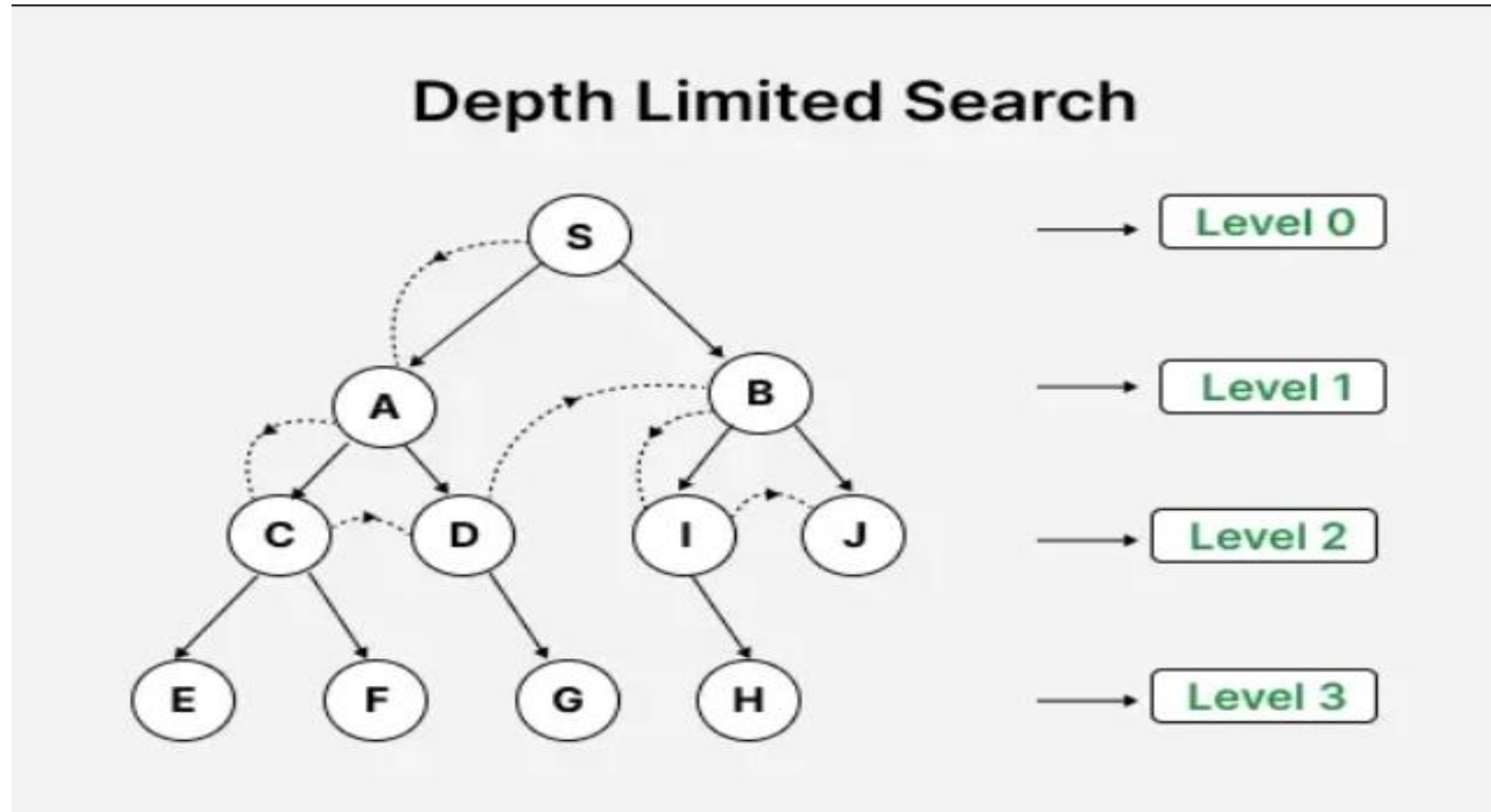
Depth Limited Search

- A key problem with Depth First Search is that it explores a graph or a tree by starting at the root node and exploring as far as possible along each branch before backtracking.
- This method can be inefficient when dealing with large or infinite trees as it may explore deep branches that do not contain the goal, leading to wasted time and resources.
- Depth Limited Search is a modified version of DFS that imposes a limit on the depth of the search.

Depth Limited Search

- Depth Limited Search will only explore nodes up to a certain depth effectively preventing it from going down excessively deep paths that may also lead to not reaching the goal and is computationally expensive.
- By setting a maximum depth limit, DLS aims to improve efficiency and ensure more manageable search times.

Depth Limited Search



Depth Limited Search

- How does it work ?
 - Initialization: Begin at the root node with a specified depth limit.
 - Exploration: Traverse the tree or graph, exploring each node's children.
 - Depth Check: If the current depth exceeds the set limit, stop exploring that path and backtrack.
 - Goal check: If the goal node is found within the depth limit, the search is successful.
 - Backtracking: If the search reaches the depth limit or a leaf node without finding the goal, backtrack and explores other branches.

Informed Search strategies

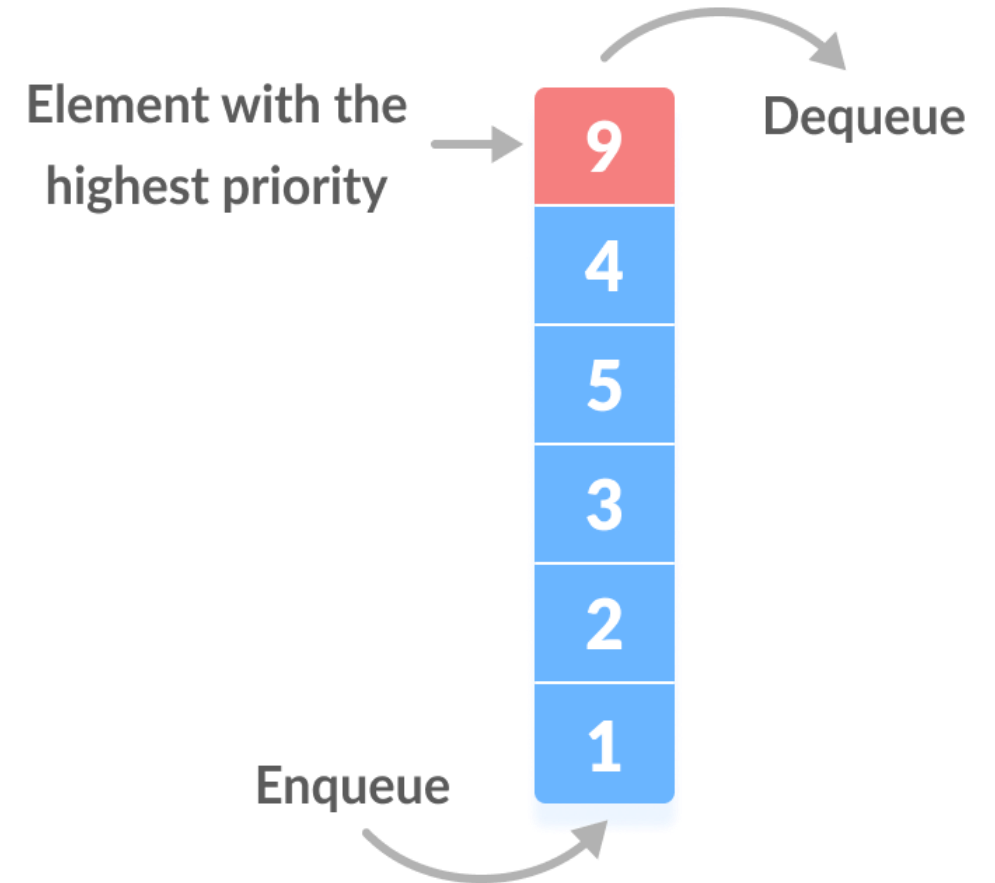
- Informed search strategy uses problem-specific knowledge beyond the problem definition itself and can find solutions more efficiently than a uniformed strategy can.
- They use additional knowledge about the problem (beyond what is given in state space) to guide the search more efficiently.
- The additional knowledge is called a heuristic $h(n)$, which estimates how close a node is to the goal.

Best First Search

- Best-first search is an instance of the general TREE-SEARCH or GRAPH-SEARCH algorithm in which a node is selected for expansion based on an evaluation function, $f(n)$.
- It selects the most promising node in the search space using the evaluation function.
- By iteratively choosing the most promising node, it aims to efficiently navigate towards the goal state, making it particularly effective for optimization problems.

Priority Queue

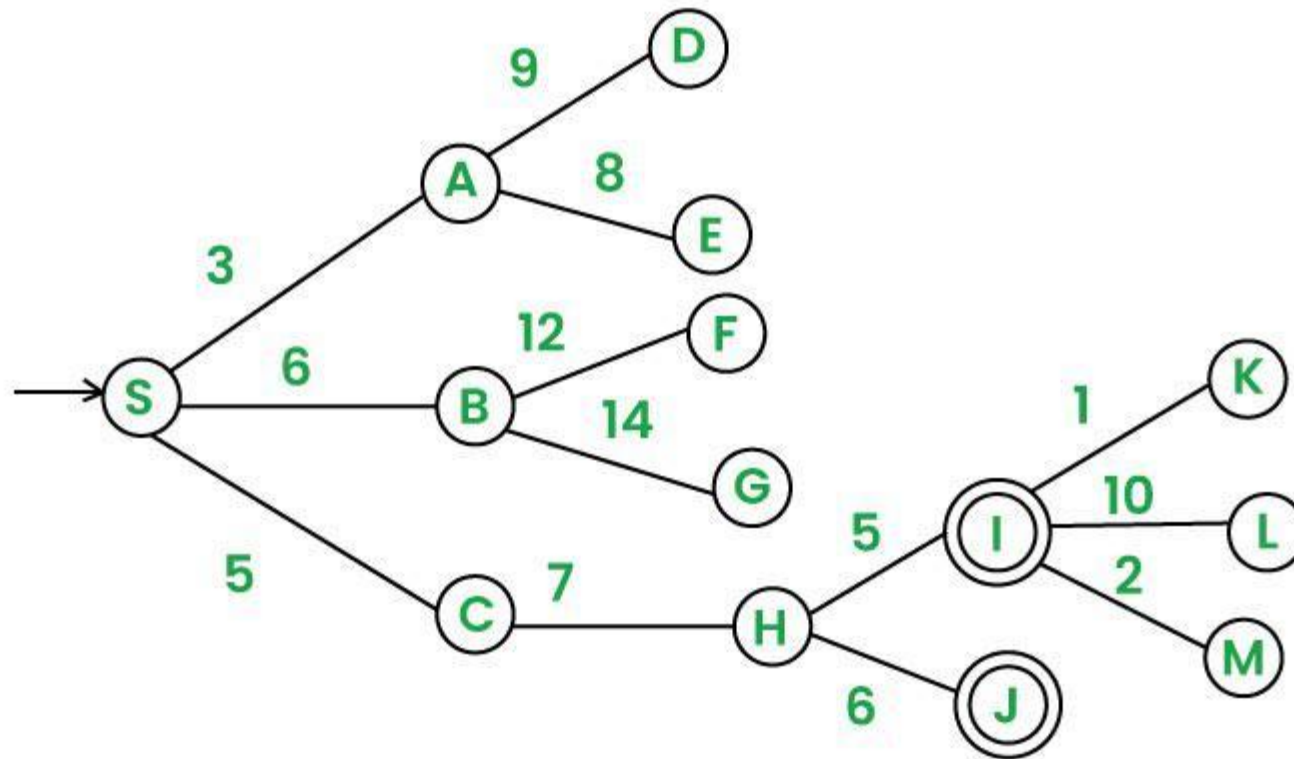
- A **priority queue** is a type of queue where each element is associated with a priority value, and elements are served based on their priority rather than their insertion order.
- Elements with higher priority are retrieved or removed before those with lower priority.
- When a new item is added, it is inserted according to its priority.



Best First Search Algorithm

- Follow the below given steps:
- Initialize an empty Priority Queue named **pq**.
- Insert the starting node into **pq**.
- While **pq** is not empty:
 - Remove the node **u** with the lowest evaluation value from **pq**.
 - If **u** is the goal node, terminate the search.
 - Otherwise, for each neighbor **v** of **u**: If **v** has not been visited, Mark **v** as visited and Insert **v** into **pq**.
 - Mark **u** as examined.
- End the procedure when the goal is reached or **pq** becomes empty.

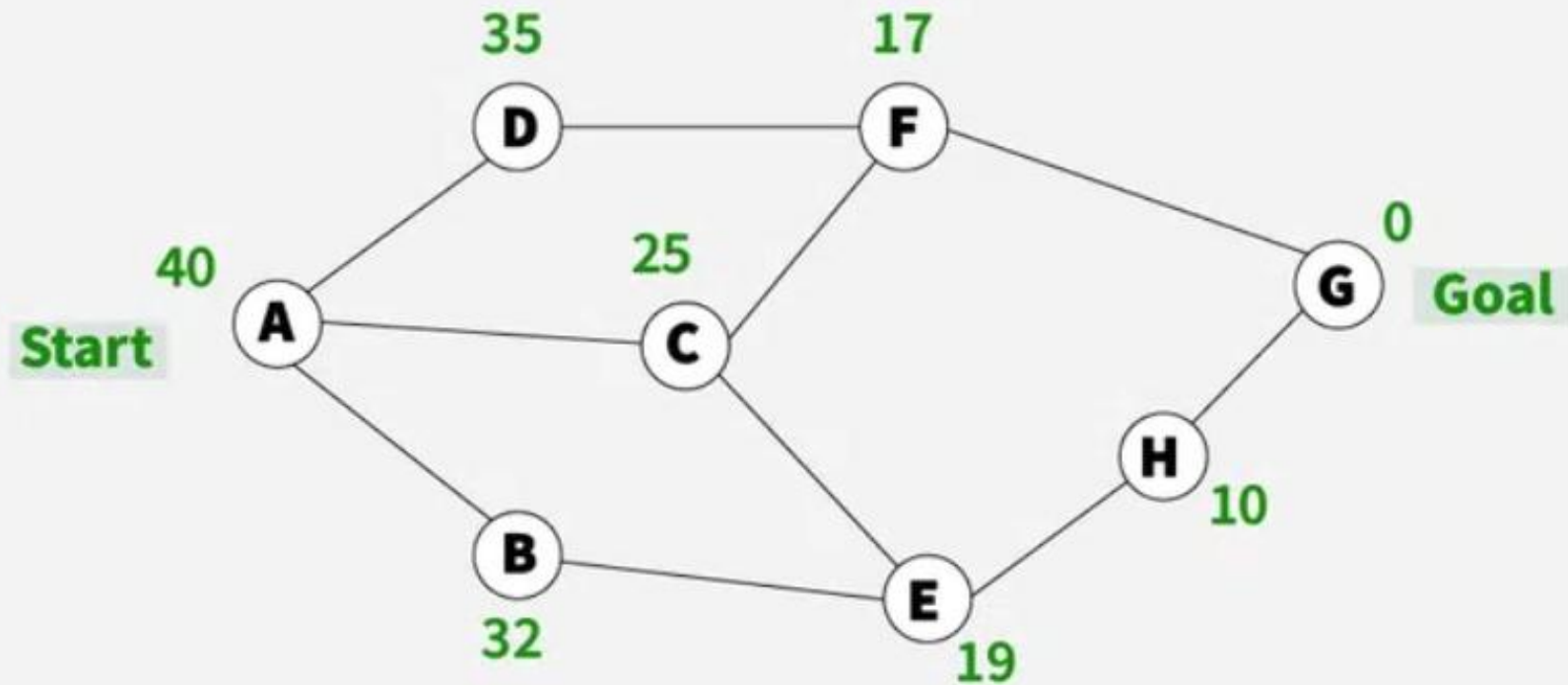
Best First Search



Best First Search

- We start from source “S” and search for goal “I” using given costs and Best First search.
- PQ (Priority Queue) initially contains S.
 - $PQ = \{S\}$
 - We remove S from PQ and process unvisited neighbors of S to pq.
 - PQ now contains $\{A(3), C(5), B(6)\}$ i.e. order in less cost.
- Remove A from PQ and process unvisited neighbors of A to pq.
 - PQ now contains $\{C(5), B(6), E(8), D(9)\}$
- Remove C from PQ and process unvisited neighbors of C to pq.
 - PQ contains $\{B(6), H(7), E(8), D(9)\}$
- Remove B from PQ and process unvisited neighbors of B to PQ.
 - PQ contains $\{H(7), E(8), D(9), F(12), G(14)\}$
- We remove H from PQ.
- Since our goal “I” is a neighbor of H, we return it.
- Path: I – H – C -- S

Classwork for Best First Search



A* search

- The most widely used best-first search is A* search.
- It evaluates nodes by combining $g(n)$, the cost to reach the node, and $h(n)$, cost to get from node to the goal:
 - $F(n) = g(n) + h(n)$
 - $g(n)$: Actual cost of reaching node n from the start node. This is the accumulated cost of the path from the start node to node n .
 - $h(n)$: The heuristic finds of the cost to reach the goal from node n . This is a weighted guess about how much further it will take to reach the goal.
- The function, $f(n) = g(n) + h(n)$ is the total estimated cost of the cheapest solution through node n

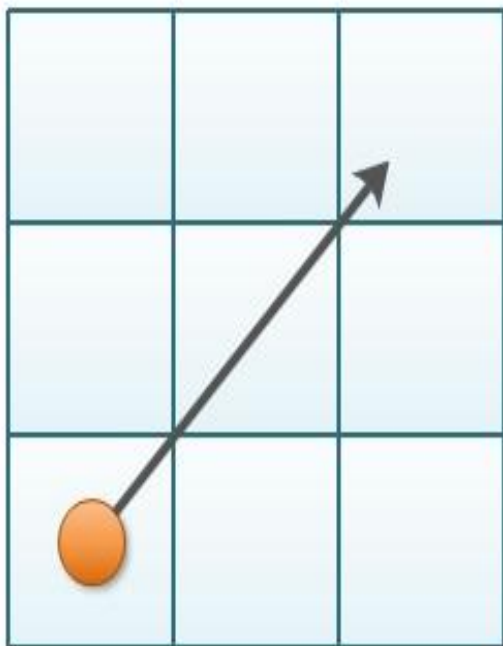
How A* algorithm works ?

- Following are the steps involved:
- **Initialization:** The initial node is added to the open set, a collection of nodes that are yet to be explored. The $f(n)$ value for the start node is calculated using the heuristic.
- **Loop:** A* selects the node with the lowest $f(n)$ value from the open set. This node is expanded and its neighbors are examined.
- **Goal Check:** If the node being processed is the goal node, the search terminates and the algorithm returns the path to the goal.
- **Node Expansion:** Each neighbor of the current node is evaluated based on the $g(n)$, $h(n)$ and $f(n)$ values. If a better path to a neighbor is found i.e a lower $f(n)$ then the neighbor is added to the open set or its values are updated.
- **Repeat:** This process continues until the goal is found or the open set is empty which means there is no solution.

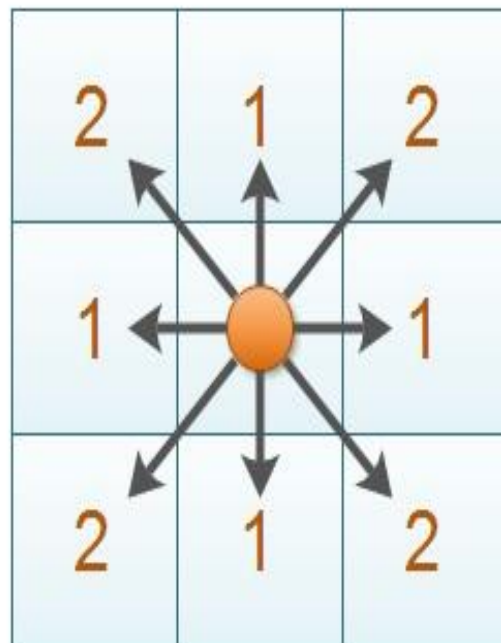
How to choose heuristic function ?

- The efficiency of the A* algorithm heavily depends on the heuristic function.
- The choice of heuristic can affect the performance and efficiency of the algorithm.
- A good heuristic function should have following properties:
- **Admissibility:** It means a heuristic never guesses a cost higher than the actual cost to the goal. This helps A* always find the shortest path. For example, using straight-line distance on a map is an admissible heuristic.
- **Consistency (Monotonicity):** It means a heuristic gives estimates that don't jump around too much. For any node and its next step, the cost to the goal should not be more than going to the next step and then to the goal. This helps A* avoid rechecking the same nodes and makes it faster.
- Commonly used heuristics:
 - Manhattan Distance
 - Euclidean Distance
 - Chebyshev Distance

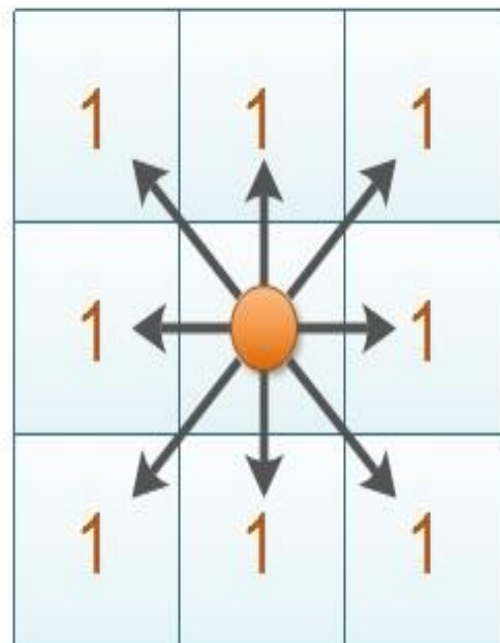
Euclidean Distance



Manhattan Distance

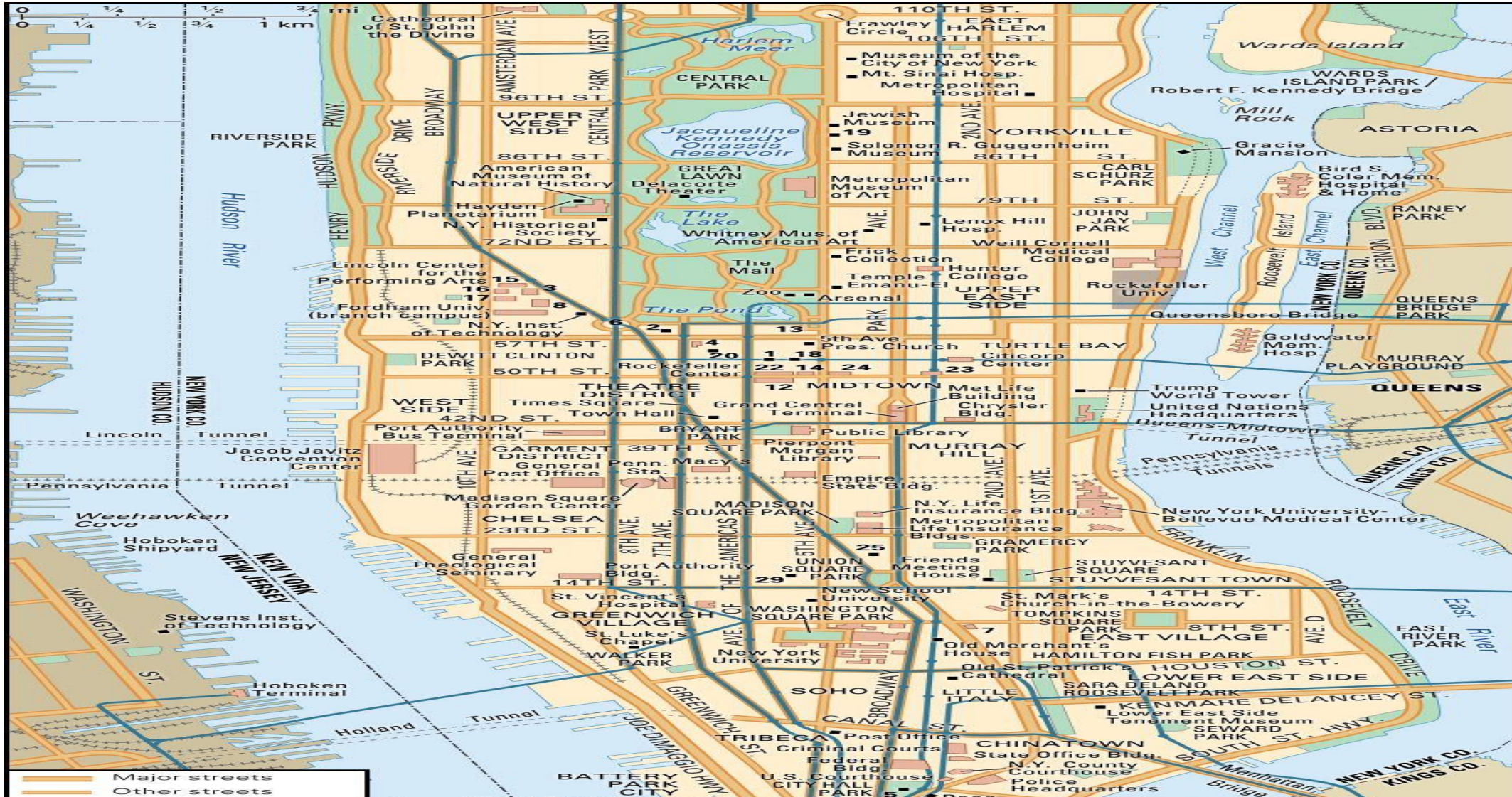


Chebyshev Distance

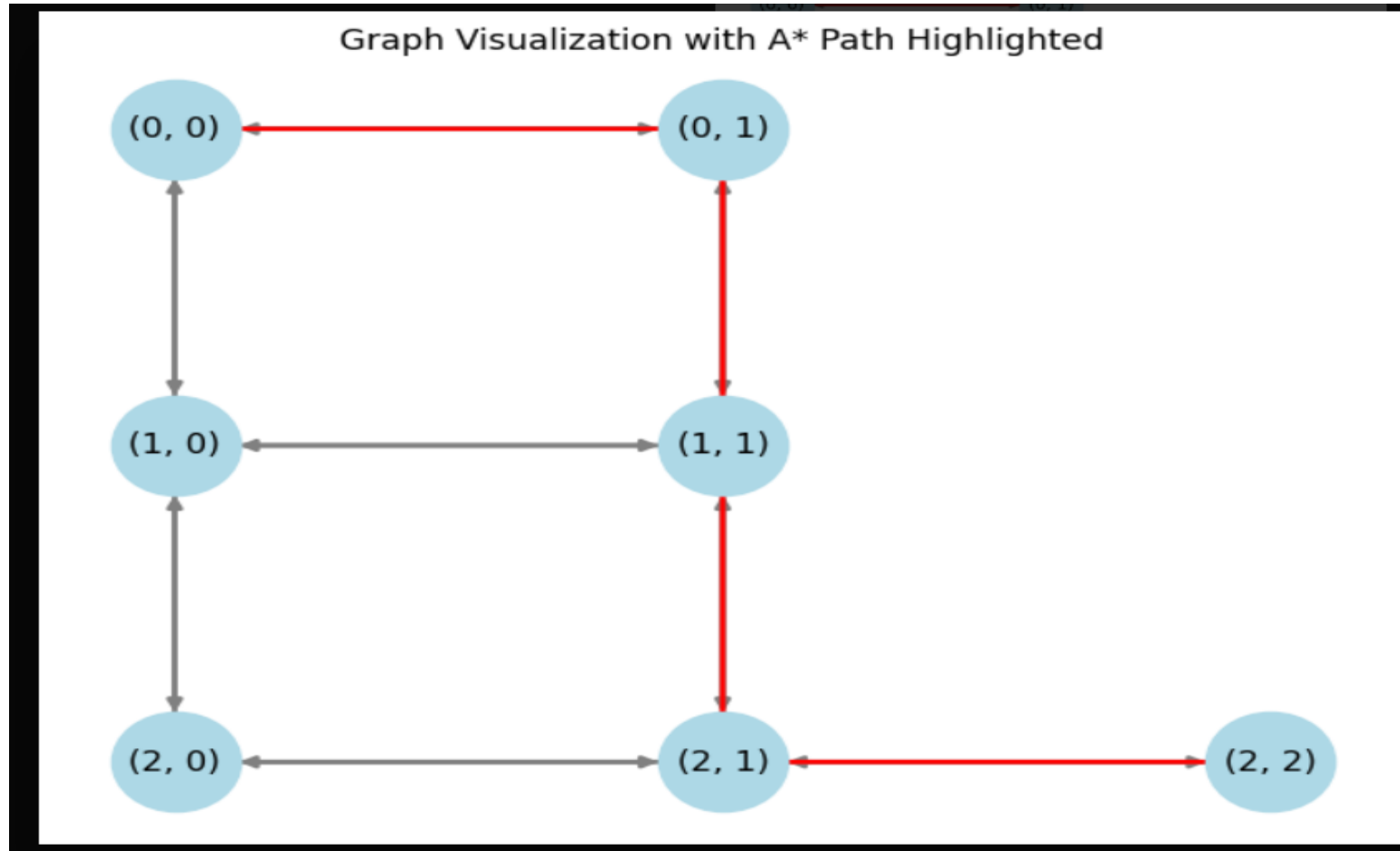


$$\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2} \quad |x_1 - x_2| + |y_1 - y_2| \quad \max(|x_1 - x_2|, |y_1 - y_2|)$$

Map of Manhattan



A* algorithm

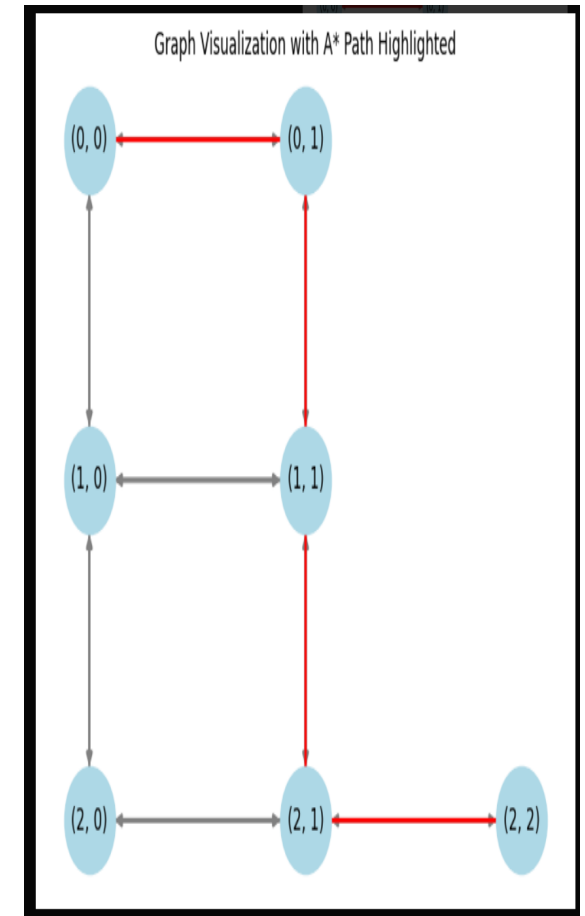


A* algorithm

- Nodes
 - Each node is a grid coordinate.
 - (0,0), (0, 1), (1,1), (2,1), (2,2)
- Start and Goal
 - Start = (0, 0)
 - Goal = (2,2)
- Edge Cost
 - Every move costs 1
- Since this is a grid, A* uses Manhattan distance.
 - $h(n) = |x(\text{goal}) - x(n)| + |y(\text{goal}) - y(n)|$

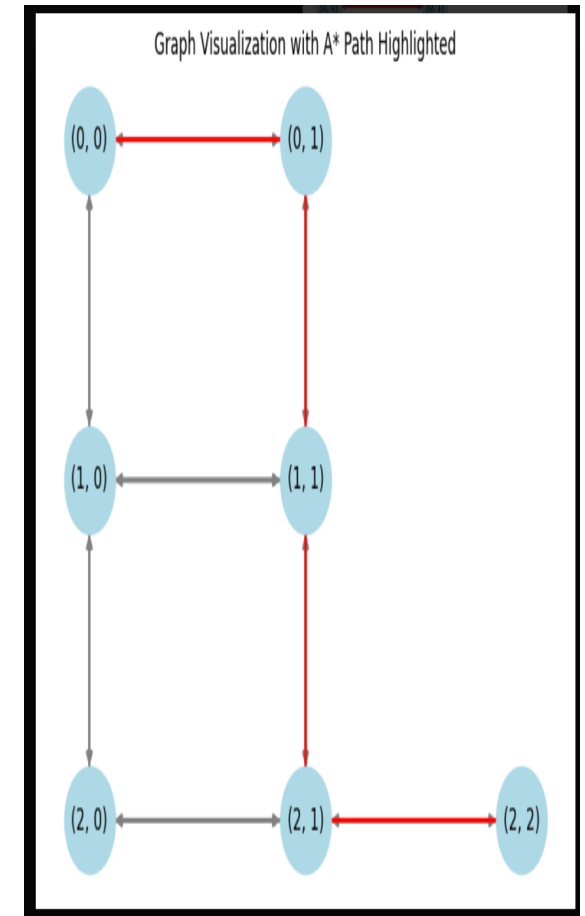
Step by Step execution

- Step 1: Start at (0,0)
 - $G(0,0) = 0$
 - $H(0,0) = |2 - 0| + |2 - 0| = 4$
 - $F(0, 0) = 0 + 4 = 4$
 - Put (0,0) in the list.
- Step 2: Expand (0,0)
 - Possible neighbors:
 - (0,1)
 - (1,0)
 - **Evaluate (0,1)**
 - $g = 1$
 - $h = |2 - 0| + |2 - 1| = 3$
 - $f = 1 + 3 = 4$
 - **Evaluate (1,0)**
 - $G = 1$
 - $H = |2 - 1| + |2 - 0| = 3$
 - $F = 1 + 3 = 4$
 - It's a case of tie and the algorithm can choose any one of those two vertices.



Step by Step execution

- Step 3: Expand (0, 1)
 - Neighbor (1,1)
 - **Evaluate (1,1)**
 - $G = 2$
 - $H = 2$
 - $F = 2 + 2 = 4$
 - Add (1,1) to the open list.
- Step 4: Expand (1,1)
 - Possible neighbors:
 - (2, 1)
 - **Evaluate (2,1)**
 - $g = 3$
 - $h = |2-2| + |2-1| = 1$
 - $f = 1 + 3 = 4$
 - Add (2,1) to the open list.
- **Step 5: Expand (2,1)**
 - **Neighbor**
 - **(2,2) Goal**
 - $G = 4$
 - $H = 0$
 - $F = 0 + 4 = 4$
 - The goal is reached in this case.



Adversarial search

- Adversarial search is a well-suited approach in a competitive environment, where two or more agents have conflicting goals.
- The adversarial search can be employed in two player zero-sum games which means each move can be good for one player and bad for another.
- Using adversarial search, AI agents can make optimal decisions while anticipating the moves of an opponents .

MiniMax algorithm

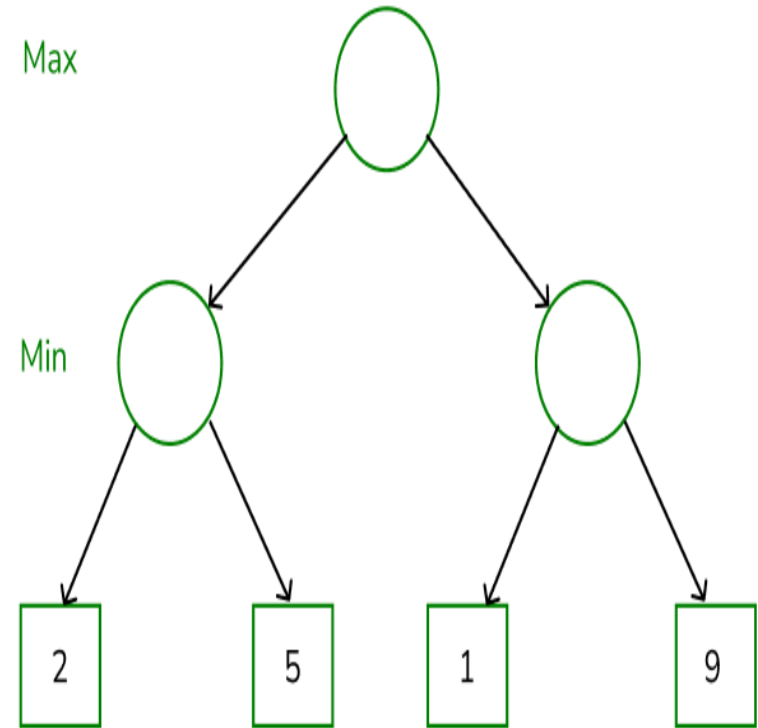
- It is also a type of adversarial search.
- It is recursive or backtracking algorithm that is responsible for choosing the best optimal move in the conflicting environment.
- It operates in a tree structure called **game tree**.
- The tree's leaves accommodate all the possible moves.
- With every single move, game state changes and game tree gets updated.
- It works by proceeding down to the tree until it reaches the leaf node and then it backs up the minimax values through the tree as the recursion unwinds.

Key terminologies in Minimax algorithm

- **Minimax Tree:** A tree structure shows all possible moves and game states in a game, used by Minimax to find the best move.
- **MAX (Maximizer)** - Maximizer seeks favorable outcomes in games by maximizing chances of winning for themselves as players.
- **MIN (Minimizer)** - Minimizer reduces winning chances, making moves leading to least favorable outcome for the Maximizer in games.
- **Initial state** - Starting state of game board, representing the configuration at the beginning of the game.
- **Terminal state** - Terminal states are the final outcomes of a game board, resulting in either a win, loss, or draw.
- **Heuristic Evaluation Function:** Function evaluates game states for maximizer, assigning numerical values to each game state based on piece positions, material advantage, and board control.

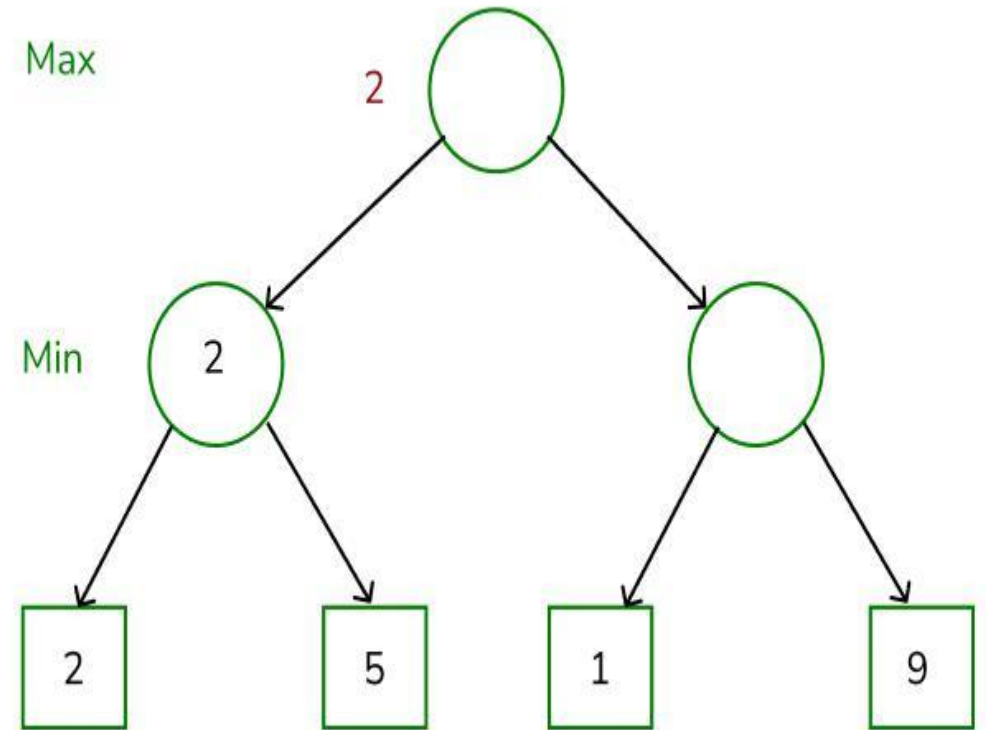
MiniMax algorithm

- Initial decision making
 - Maximizer has option to choose between two paths(L/R)
- Minimizer's choice
 - Assuming Maximizer traverses to the left where it reaches minimizer node, the minimizer has responsibility of selecting between (2,5).
 - It has to minimize the maximizer's score and hence chooses 2.



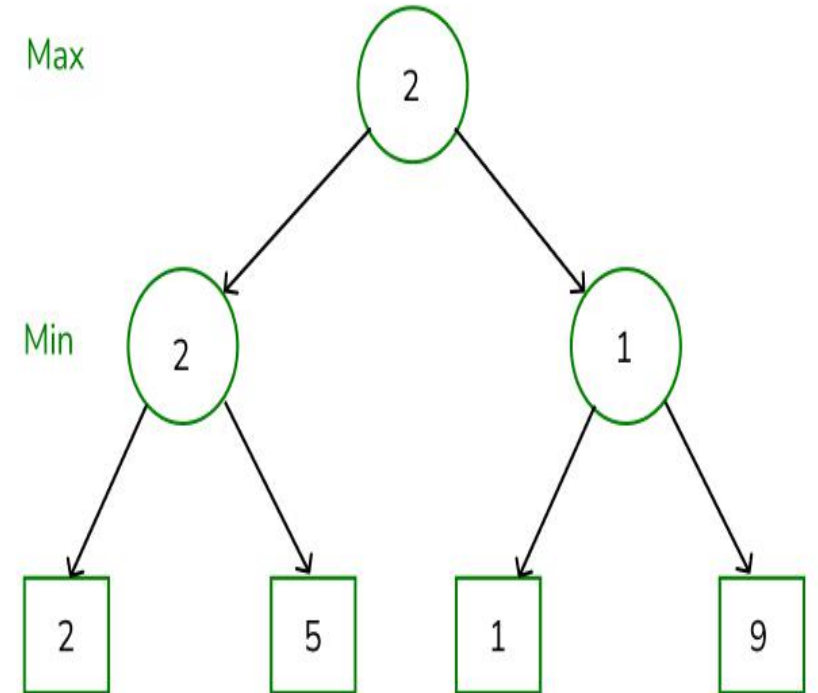
MiniMax algorithm

- Traversal to the right
 - Maximizer then traverses to the right side of the tree and encounters other set of nodes.
 - It reaches minimizer where the minimizer again selects the optimal minimum value 1 among its options.
 - The process continues recursively where each player makes decisions based on whether they are maximizer or minimizer.



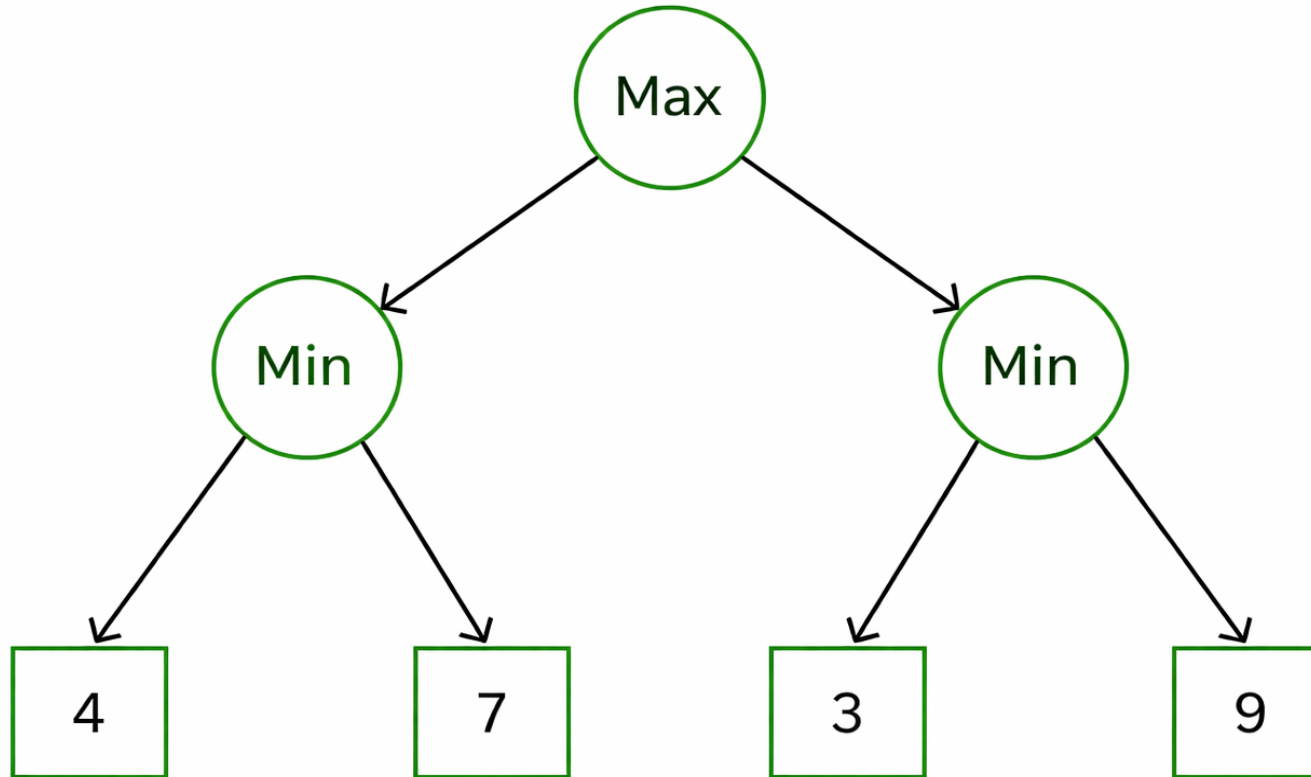
MiniMax algorithm

- Comparison and outcome
 - The maximizer node (root node) already holds the value of 2, when the player earns another optimal value that is 1 while traversing in the right path, it is then compared to get the final optimal value.
 - The maximizer earns the optimal value 2 based on the decisions made by both players.



Classwork MiniMax algorithm

Classwork: Solve the Minimax Problem



Problems with MiniMax algorithm

- Minimax algorithm is effective in competitive scenario for agents.
- On the other hand, it can be computationally expensive, particularly for games like chess.
- The time complexity of MiniMax is $O(b^D)$,
 - Where, b = branching factor,
 - D = depth of the tree.
 - The exponential complexity makes the algorithm impractical for deep searches without optimization techniques.

Alpha Beta pruning

- Alpha Beta pruning is an optimization technique for the minimax algorithm.
- It reduces the number of nodes evaluated in the game tree by eliminating branches that cannot influence the final decision.
- This is done by maintaining two variables i.e. alpha and beta.
 - Alpha : - The best(maximum) value that the maximizer can guarantee given the current state.
 - Beta : - The best (minimum) value that the minimizer can guarantee given the current state.

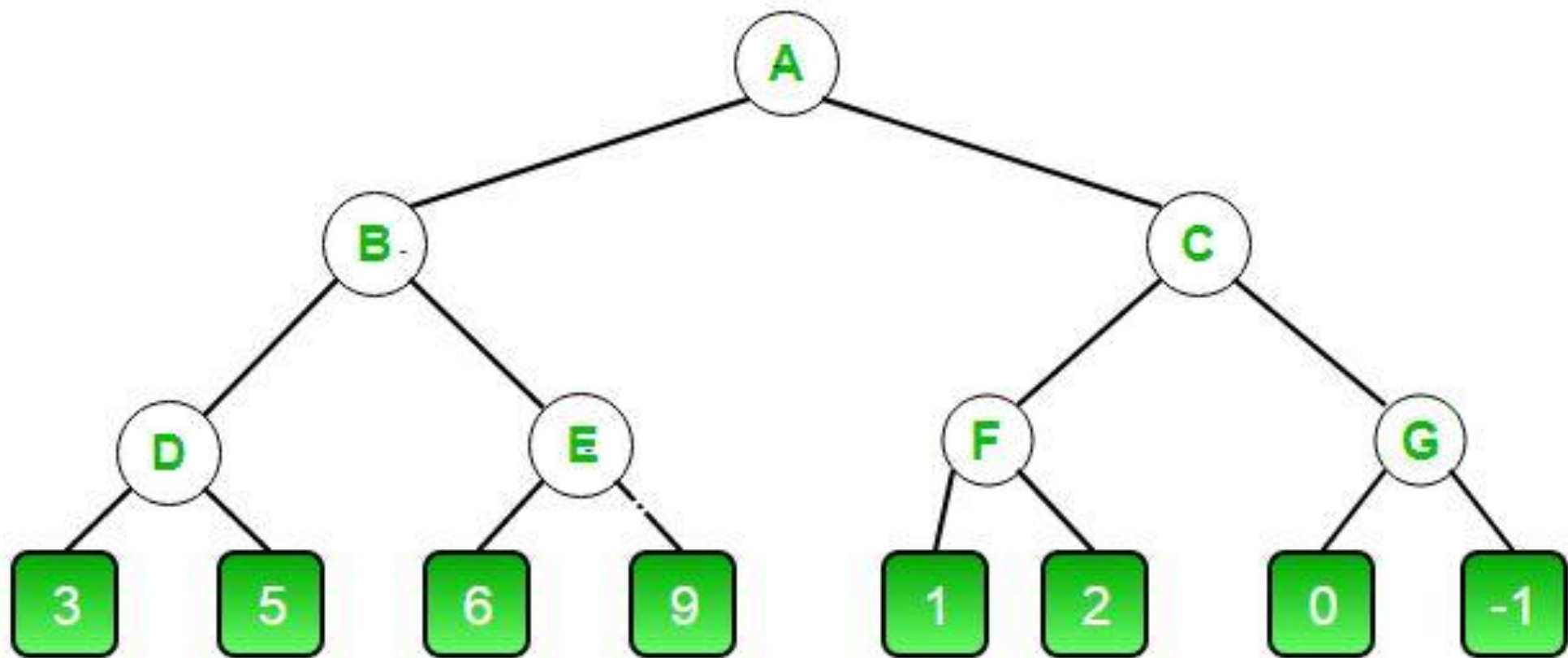
Steps Involved Alpha-Beta Pruning

- Initialization: Start with alpha set to negative infinity and beta set to positive infinity.
- Max Node Evaluation:
 - For each child of Max node:
 - Evaluate the child node using the Minimax algorithm with Alpha-Beta pruning.
 - Update $\alpha = \max(\alpha, \text{child value})$
 - If α is greater than or equal to β , prune the remaining children (beta cutoff).
- Min Node Evaluation
 - For each child of a Min node:
 - Evaluate the child node using the Minimax algorithm with Alpha-Beta pruning.
 - Update β : $\beta = \min(\beta, \text{childvalue})$
 - If β is less than or equal to α , prune the remaining children (alpha cutoff).

MAX

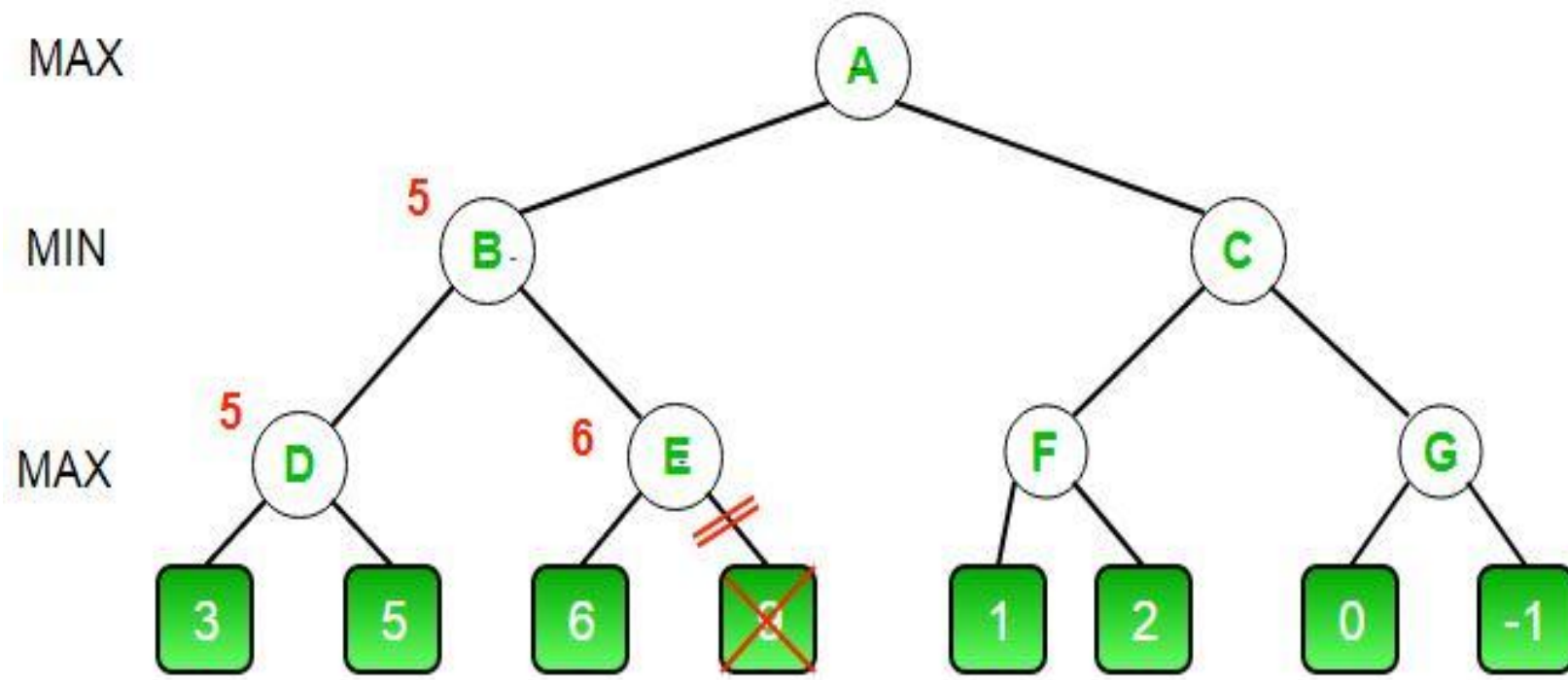
MIN

MAX



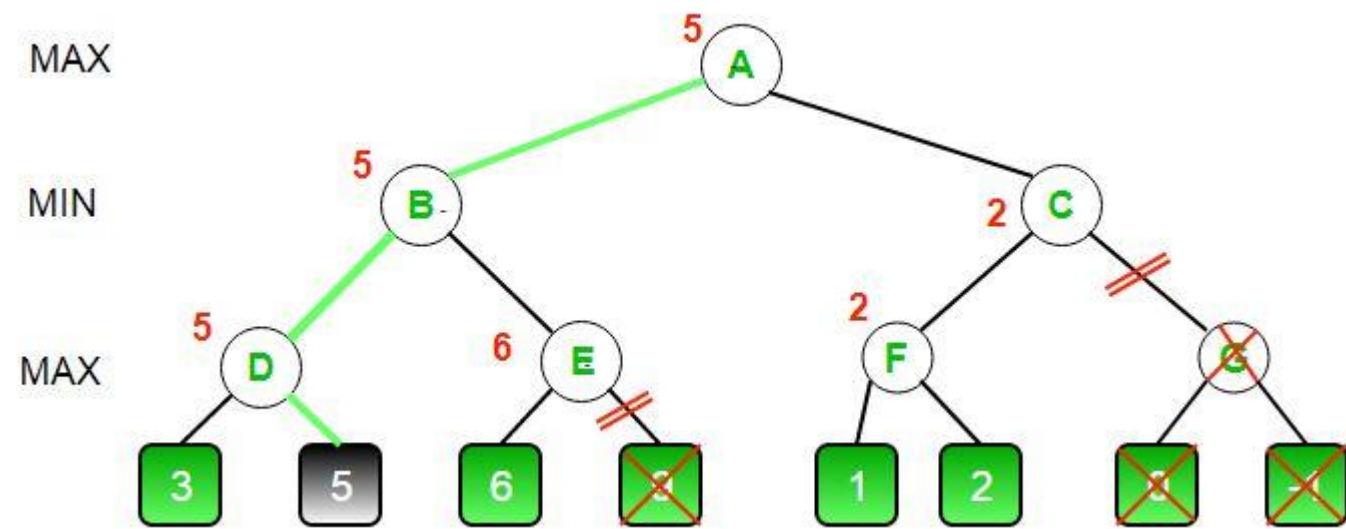
Alpha Beta pruning

- The initial call starts from **A**. The value of alpha here is **-INFINITY** and the value of beta is **+INFINITY**. These values are passed down to subsequent nodes in the tree. At **A** the maximizer must choose max of **B** and **C**, so **A** calls **B** first
- At **B** it the minimizer must choose min of **D** and **E** and hence calls **D** first.
- At **D**, it looks at its left child which is a leaf node. This node returns a value of 3. Now the value of alpha at **D** is $\max(-\text{INF}, 3)$ which is 3.
- To decide whether its worth looking at its right node or not, it checks the condition $\beta \leq \alpha$. This is false since $\beta = +\text{INF}$ and $\alpha = 3$. So, it continues the search.
- **D** now looks at its right child which returns a value of 5. At **D**, $\alpha = \max(3, 5)$ which is 5. Now the value of node **D** is 5
- **D** returns a value of 5 to **B**. At **B**, $\beta = \min(+\text{INF}, 5)$ which is 5. The minimizer is now guaranteed a value of 5 or lesser. **B** now calls **E** to see if he can get a lower value than 5.
- At **E** the values of alpha and beta is not $-\text{INF}$ and $+\text{INF}$ but instead $-\text{INF}$ and 5 respectively, because the value of beta was changed at **B** and that is what **B** passed down to **E**
- Now **E** looks at its left child which is 6. At **E**, $\alpha = \max(-\text{INF}, 6)$ which is 6. Here the condition becomes true. β is 5 and α is 6. So, $\beta \leq \alpha$ is true. Hence it breaks and **E** returns 6 to **B**.
- Note how it did not matter what the value of **E**'s right child is. It could have been $+\text{INF}$ or $-\text{INF}$, it still wouldn't matter, We never even had to look at it because the minimizer was guaranteed a value of 5 or lesser.
- **E** returns a value of 6 to **B**. At **B**, $\beta = \min(5, 6)$ which is 5. The value of node **B** is also 5



Alpha Beta Pruning Contd.

- **B** returns 5 to **A**. At **A**, $\alpha = \max(-\text{INF}, 5)$ which is 5. Now the maximizer is guaranteed a value of 5 or greater. **A** now calls **C** to see if it can get a higher value than 5.
- At **C**, $\alpha = 5$ and $\beta = +\text{INF}$. **C** calls **F**.
- At **F**, $\alpha = 5$ and $\beta = +\text{INF}$. **F** looks at its left child which is a 1. $\alpha = \max(5, 1)$ which is still 5.
- **F** looks at its right child which is a 2. Hence the best value of this node is 2. Alpha still remains 5
- **F** returns a value of 2 to **C**. At **C**, $\beta = \min(+\text{INF}, 2)$. The condition $\beta \leq \alpha$ becomes true as $\beta = 2$ and $\alpha = 5$. So, it breaks and it does not even have to compute the entire sub-tree of **G**.
- The intuition behind this break-off is that, at **C** the minimizer was guaranteed a value of 2 or lesser. But the maximizer was already guaranteed a value of 5 if he choose **B**. So why would the maximizer ever choose **C** and get a value less than 2 ? Again, you can see that it did not matter what those last 2 values were. We also saved a lot of computation by skipping a whole sub-tree.
- **C** now returns a value of 2 to **A**. Therefore, the best value at **A** is $\max(5, 2)$ which is a 5.
- Hence the optimal value that the maximizer can get is 5.



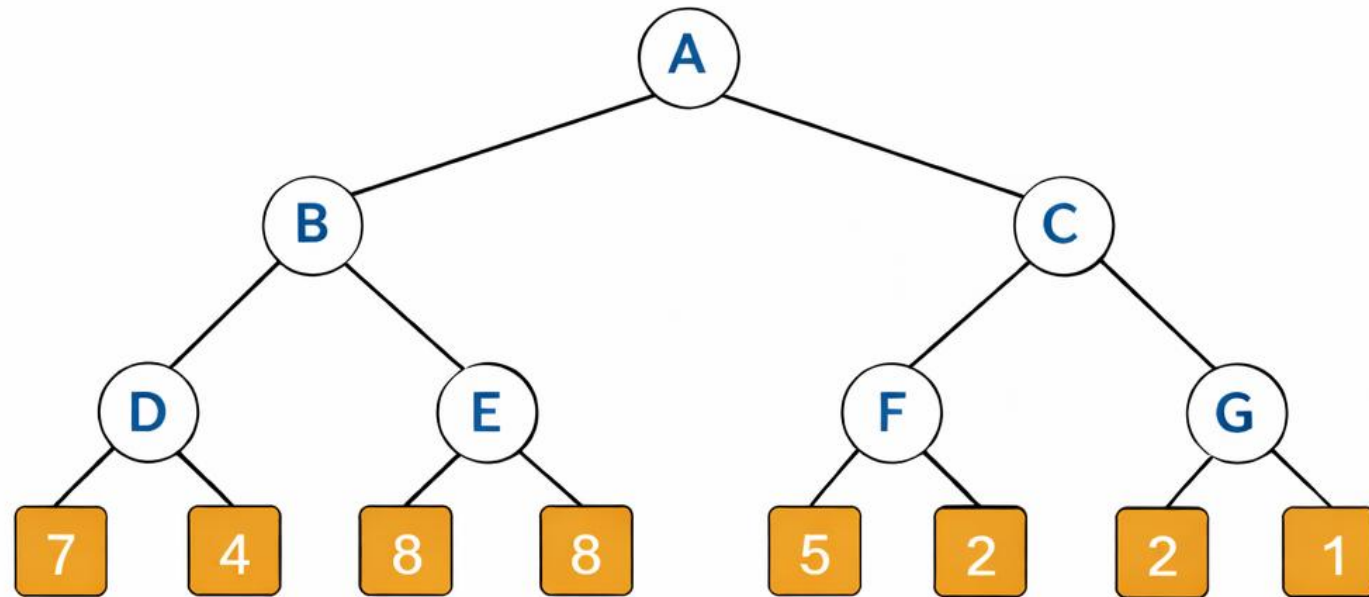
Classwork: Alpha-Beta Pruning

Solve this game tree using alpha-beta pruning. Write the final value of the root.
Cross out any branches that are pruned. Show α and β values as you traverse.

Max nodes maximize their value (update α).

Min nodes minimize their value (update β).

Prune branches where $\alpha \geq \beta$ for Max nodes or $\beta \leq \alpha$ for Min nodes.



1. Pruned branches: _____ Final value of root A: _____

THANK YOU !!!!!!!!!!!!!!!